



Concurrency Control

Talia Noreen

**Beaconhouse National University
School of Computer and Information Technology (SCIT)**

Introduction

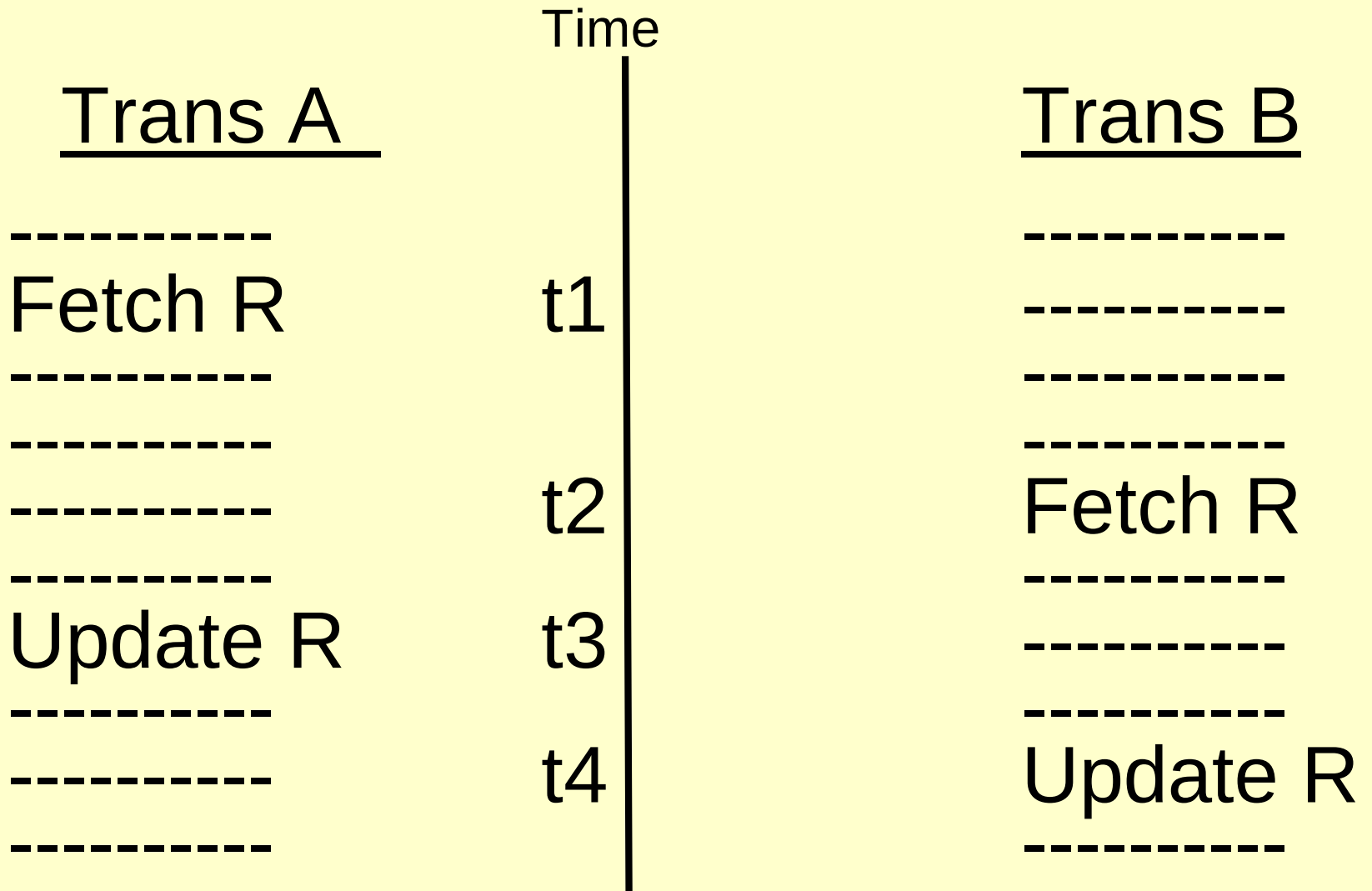
- Database Concurrency refers to database access by multiple transactions (**Concurrent Transactions**).
- A **Concurrency Control** mechanism is needed to prevent loss of data integrity due to interference between users (concurrent operations) in an multi-user environment.
- Concurrency control is a process of managing simultaneous operations on the database without having them interfere with one another.
- When two or more users are accessing the database simultaneously and at least one is updating data, there may be interference that can result in inconsistencies.

Concurrency Problems

- The concurrent transactions yield to the following problems:
 1. Lost Update Problem
 2. Uncommitted Dependency Problem
 3. Inconsistent Retrievals

(10-2 c, Text Book P-492), (Ex 22-3, RB-B P-674)

1. Lost Update Problem



1. Lost Update Problem

Time	T_1	T_2	bal_x
t_1		begin_transaction	100
t_2	begin_transaction	read(bal_x)	100
t_3	read(bal_x)	$bal_x = bal_x + 100$	100
t_4	$bal_x = bal_x - 10$	write(bal_x)	200
t_5	write(bal_x)	commit	90
t_6	commit		90

1. Lost Update Problem

- Trans A retrieves a record R at t1.
- Trans B retrieves the same record at t2.
- Trans A updates the record R at t3.
- Trans B updates the same record at time t4.
- Transaction A's update is lost at time t4, because transaction B overwrites it.

2. Uncommitted Dependency (or dirty read) Problem

- It arises if one transaction is allowed to retrieve (or worse, update) a record that has been updated by another transaction but has not yet been committed by that other transaction.
- In the Fig, Tans A will be operating on false assumption which may lead to a chain of erroneous operations.

2. Uncommitted Dependency Problem

Time

Trans A

Trans B

Fetch R

t1

t2

t3

Update R

Rollback

2. Uncommitted Dependency Problem

Time

Trans A

Trans B

Update R

t1

t2

t3

Update R

Rollback

2. Uncommitted Dependency Problem

Time	T ₃	T ₄	bal _x
t ₁		begin_transaction	100
t ₂		read(bal _x)	100
t ₃		bal _x = bal _x + 100	100
t ₄	begin_transaction	write(bal _x)	200
t ₅	read(bal _x)	:	200
t ₆	bal _x = bal _x - 10	rollback	100
t ₇	write(bal _x)		190
t ₈	commit		190

Scheduler

- DBMS manages the concurrent transactions in a problem free manner using a built-in **scheduler**.
- **Scheduler** establishes order of concurrent transaction execution to ensure **serializability** and isolation of transactions.
- The scheduler's main job is to create a serializable schedule in which the interleaved execution of the transactions yields the same results as if the transactions were executed in serial order (one after another).
- **Serial vs. Non-serial schedule!**

Scheduler

- Serial schedule is a schedule where the operations of each transaction are executed consecutively without any interleaved operations from other transactions. However, there is no guarantee that the results of all serial executions of a given set of transactions will be identical.
- Non-Serial schedule is a schedule where the operations from a set of concurrent transactions are interleaved. Every serial execution is considered correct!
- The (non-serial) schedule is correct if it produces the same result as some serial execution. Such a schedule is called serializable.
- The objective of serializability is to find nonserial schedules that allow transactions to execute concurrently without interfering with one another, and thereby produce a database state that could be produced by a serial execution.

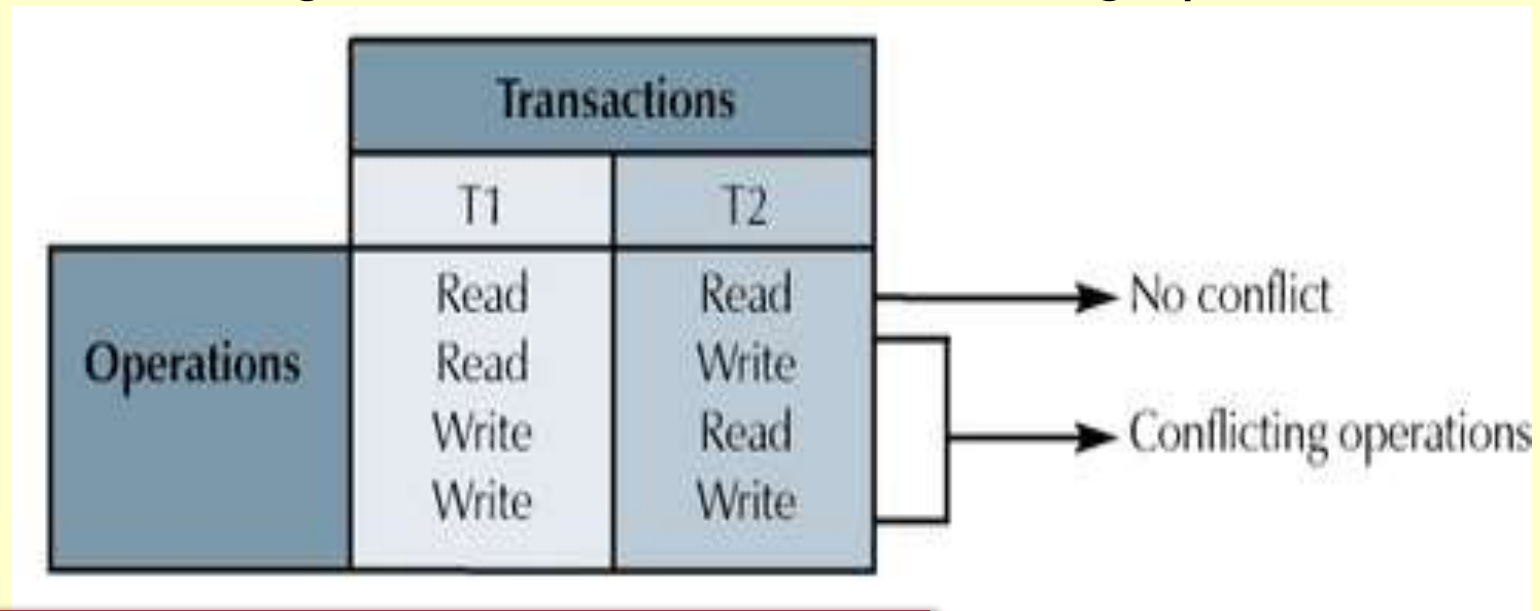
Concurrency Control Methods

- Following approaches are used for concurrency control:
 1. Pessimistic Approach (Locking)
 2. Optimistic Approach (Versioning or Time stamping)

Concurrency Control Methods

1. Locking:

- The basic idea is that when a transaction needs some object, it acquires a lock on that object.
- The effect of Lock is to “Lock other transactions out of” the object and thus prevent them from intervening the operation.
- The following matrix shows the conflicting operations:



Locking Levels

- DBMS automatically initiates and enforces locking procedures managed by **lock manager (built-in DBMS)**
- The **Locking Level (also called Granularity)** is the extent of the database resource that is included with each lock.
- There can be the following Locking levels (arranged from most restrictive to least restrictive):
 1. Database
 2. Table
 3. Page Level (Latch)
 4. Record Level
 5. Field Level

Types of Locks

1. Shared Locks
2. Exclusive Locks

1. Shared Locks:

- These are also called **S Locks** or **Read Locks**.
- These allow other transactions to read (but not update) a locked object.
- A transaction should place S Lock on a record, when it will only read(but not update) that record.
- Placing a S Lock on a record prevents other users from placing exclusive lock on it.

2. Exclusive Locks

- These are also called **X Locks** or **Write Locks**.
- These prevent other transactions to read (and therefore update) a locked object.
- A transaction should place X Lock on a record, when it is about to update that record.
- Placing a X Lock on a record prevents other users from any type of lock on it.
- It is only when the X lock has been released that the effects of the write operation will be made visible to other transactions.

Types of Locks

-	X	S	-
X	N	N	Y
S	N	Y	Y
-	Y	Y	Y

Two-phase Locking (2PL)

- Using locks in transactions, does not guarantee serializability of schedules by themselves, as shown below:

Time	T ₉	T ₁₀
t ₁	begin_transaction	
t ₂	read(bal_x)	
t ₃	bal_x = bal_x + 100	
t ₄	write(bal_x)	begin_transaction
t ₅		read(bal_x)
t ₆		bal_x = bal_x * 1.1
t ₇		write(bal_x)
t ₈		read(bal_y)
t ₉		bal_y = bal_y * 1.1
t ₁₀		write(bal_y)
t ₁₁	read(bal_y)	commit
t ₁₂	bal_y = bal_y - 100	
t ₁₃	write(bal_y)	
t ₁₄	commit	

Two-phase Locking (2PL)

$S = \{\text{write_lock}(T_9, \text{bal}_x), \text{read}(T_9, \text{bal}_x), \text{write}(T_9, \text{bal}_x), \text{unlock}(T_9, \text{bal}_x),$
 $\text{write_lock}(T_{10}, \text{bal}_x), \text{read}(T_{10}, \text{bal}_x), \text{write}(T_{10}, \text{bal}_x), \text{unlock}(T_{10}, \text{bal}_x),$
 $\text{write_lock}(T_{10}, \text{bal}_y), \text{read}(T_{10}, \text{bal}_y), \text{write}(T_{10}, \text{bal}_y), \text{unlock}(T_{10}, \text{bal}_y),$
 $\text{commit}(T_{10}), \text{write_lock}(T_9, \text{bal}_y), \text{read}(T_9, \text{bal}_y), \text{write}(T_9, \text{bal}_y),$
 $\text{unlock}(T_9, \text{bal}_y), \text{commit}(T_9)\}$

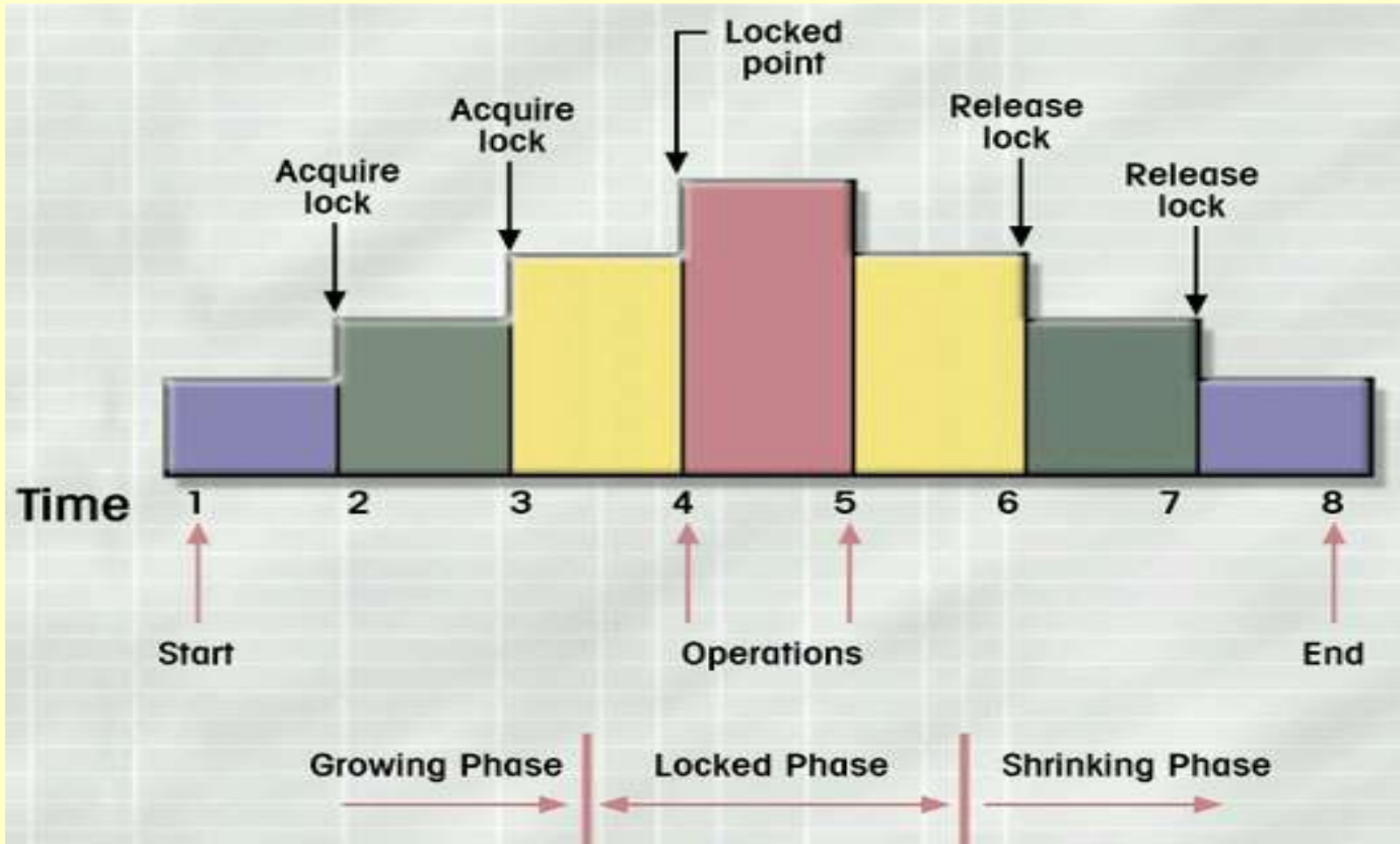
If, prior to execution, $\text{bal}_x = 100$, $\text{bal}_y = 400$, the result should be $\text{bal}_x = 220$, $\text{bal}_y = 330$, if T_9 executes before T_{10} , or $\text{bal}_x = 210$ and $\text{bal}_y = 340$, if T_{10} executes before T_9 . However, the result of executing schedule S would give $\text{bal}_x = 220$ and $\text{bal}_y = 340$. (S is **not** a serializable schedule.)

- The problem in this example is that the schedule releases the locks that are held by a transaction as soon as the associated read/write is executed.

Two-phase Locking (2PL)

- To guarantee serializability, the best known protocol is **two-phase locking (2PL)**.
- A transaction follows the two-phase locking protocol if all locking operations precede the first unlock operation in the transaction.
- According to the above protocol, every transaction can be divided into two phases: **growing phase** and **shrinking phase**.
- In **growing phase**, a transaction acquires all the locks needed by it but it cannot release any locks. Once all the locks have been acquired, the transaction is in its **locked point or locked phase**.
- In **shrinking phase**, a transaction releases its locks but cannot acquire any new locks.

Two-phase Locking



Preventing Lost Update using 2PL

Time	T ₁	T ₂	bal _x
t ₁		begin_transaction	100
t ₂	begin_transaction	read(bal _x)	100
t ₃	read(bal _x)	bal _x = bal _x + 100	100
t ₄	bal _x = bal _x - 10	write(bal _x)	200
t ₅	write(bal _x)	commit	90
t ₆	commit		90

Time	T ₁	T ₂	bal _x
t ₁		begin_transaction	100
t ₂	begin_transaction	write_lock(bal _x)	100
t ₃	write_lock(bal _x)	read(bal _x)	100
t ₄	WAIT	bal _x = bal _x + 100	100
t ₅	WAIT	write(bal _x)	200
t ₆	WAIT	commit/unlock(bal _x)	200
t ₇	read(bal _x)		200
t ₈	bal _x = bal _x - 10		200
t ₉	write(bal _x)		190
t ₁₀	commit/unlock(bal _x)		190

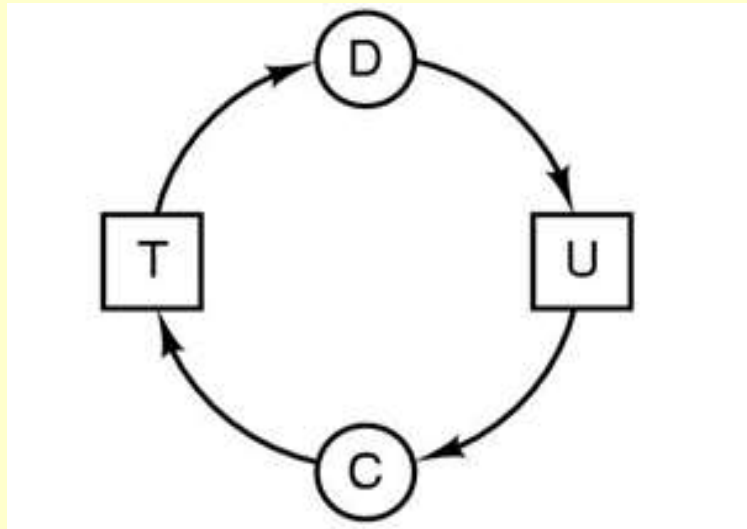
Preventing Uncommitted Dependency using 2PL

Time	T ₃	T ₄	bal _x
t ₁		begin_transaction	100
t ₂		read(bal _x)	100
t ₃		bal _x = bal _x + 100	100
t ₄	begin_transaction	write(bal _x)	200
t ₅	read(bal _x)	:	200
t ₆	bal _x = bal _x - 10	rollback	100
t ₇	write(bal _x)		190
t ₈	commit		190

Time	T ₃	T ₄	bal _x
t ₁		begin_transaction	100
t ₂		write_lock(bal _x)	100
t ₃		read(bal _x)	100
t ₄	begin_transaction	bal _x = bal _x + 100	100
t ₅	write_lock(bal _x)	write(bal _x)	200
t ₆	WAIT	rollback/unlock(bal _x)	100
t ₇	read(bal _x)		100
t ₈	bal _x = bal _x - 10		100
t ₉	write(bal _x)		90
t ₁₀	commit/unlock(bal _x)		90

Deadlock

- Locking does solve the concurrency problems, but leads to another problem called **Deadlock**.
- Deadlock is a situation in which two or more transactions have locked a common resource, and each must wait for the other to unlock that resource.



Deadlock Scenario

Time

Trans A

Trans B

Lock X

Request X

(Wait for Y)

t1

t2

t3

t4

Lock Y

Request Y
(Wait for X)

Deadlock!

Managing Deadlock

- There are the following techniques for handling deadlock:
 1. Timeouts
 2. Deadlock Prevention
 3. Deadlock Avoidance
 3. Deadlock detection and recovery

Managing Deadlock

1. Timeouts

- In this approach, a transaction that requests a lock will wait for only a system-defined period of time. If the lock has not been granted within this period, the lock request times out.
- In this case, the DBMS assumes that the transaction may be deadlocked, even though it may not be, and it aborts and automatically restarts the transaction.

2. Deadlock Avoidance

- A transaction obtains all its locks when it begins, or it waits until all the locks are available (**conservative 2PL**). Not very practical as it is often difficult to predict all the required locks in advance.

3. Deadlock Prevention:

- The DBMS looks ahead to determine if a transaction would cause deadlock, and never allows deadlock to occur. A transaction requesting a new lock is aborted when there is the possibility that a deadlock can occur.

Managing Deadlock

4. Deadlock Detection and Recovery:

- A involves detecting a deadlock and then breaking it.
- The DBMS maintains a **matrix of resource usage** or a **Wait-For Graph (WFG)**, which maintains “Who is Waiting for Whom”.
- Deadlock exists if and only if the WFG contains a cycle.
- Breaking the deadlock involves choosing one of the deadlocked transactions as a **victim** and rolling it back, thereby releasing all its locks and allowing other transaction to proceed.
- The following issues need to be considered:
 - What should be the frequency of invoking deadlock detection algorithm? (Fixed or Static/ Variable or Dynamic)!
 - Choice of deadlock victim
 - How far to roll a transaction back?
 - Avoiding starvation

Livelock

- It is also possible for transactions to be in **livelock**, that is, left in a wait state indefinitely, unable to acquire any new locks, although the DBMS is not in deadlock.
- This can happen if the waiting algorithm for transactions is unfair and does not take account of the time that transactions have been waiting.
- To avoid livelock, a priority system can be used, whereby the longer a transaction has to wait, the higher its priority.

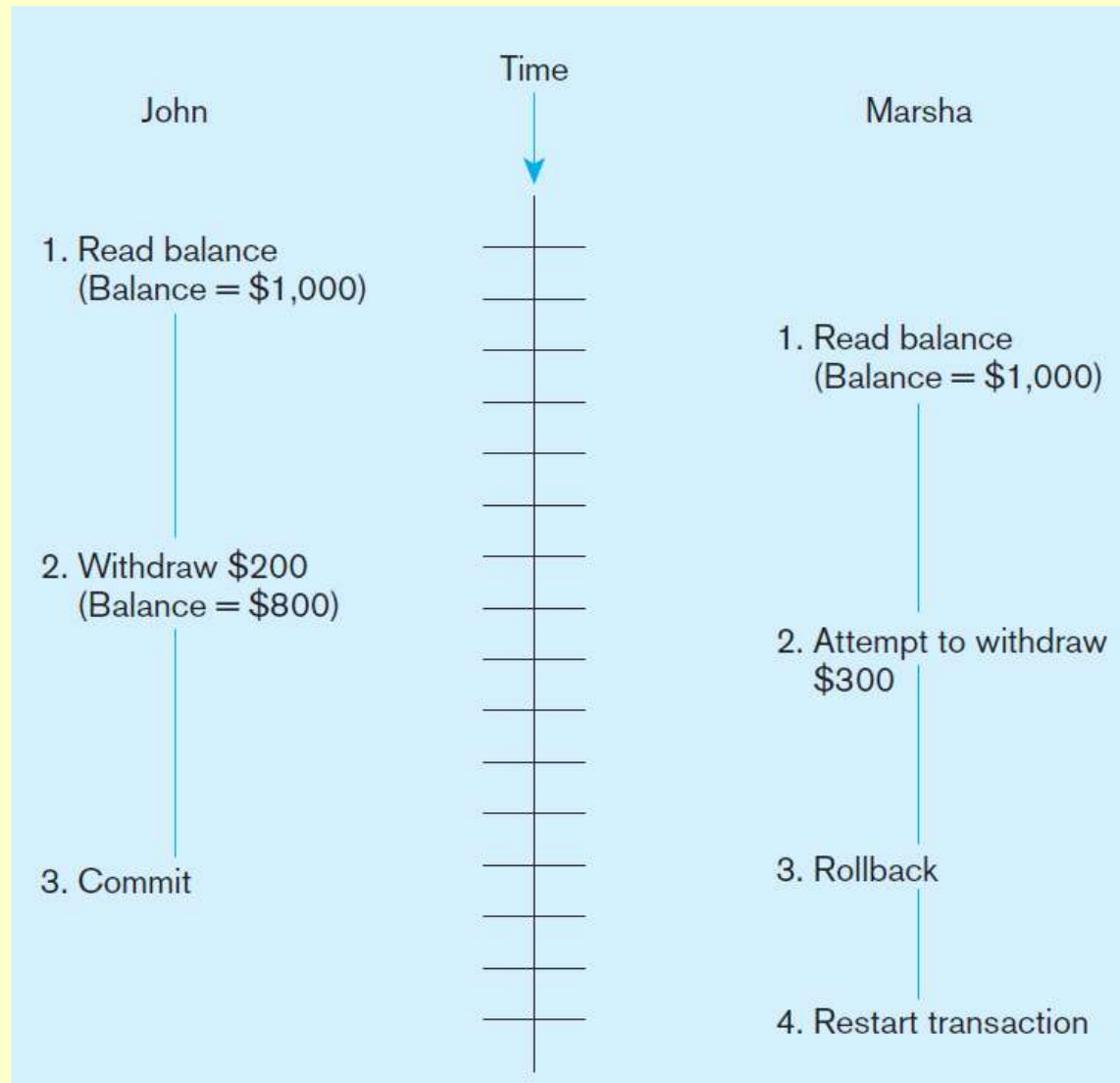
Versioning or Time Stamping Method

- With Optimistic Approach (Versioning) there is no form of locking.
- It assumes that most of the time, two or more transactions will not be using the same object.
- Time stamp is a **unique identifier** that indicates the relative starting time of a transaction.
- **Time stamping** is a concurrency protocol that orders the transactions in such a way that older transactions (transactions with smaller time stamps) get priority in the event of a conflict.
- Each transaction is restricted to a view of the database as it is at the start of the transaction, and when a transaction modifies a record, the DBMS, creates a new record version instead of overwriting the old record.
- The DBMS provides the requested records to the transactions with a timestamp on it.

Versioning or Time Stamping Method

- Suppose that two transactions modify the same record in different/conflicting ways, the update of the earlier (older) transaction will be accepted and that of later (younger) transaction will be rolled back and then restarted with a new time stamp value.
- The main advantage of versioning over locking is performance improvement.

Versioning or Time Stamping Method



Thank you for your attention.

Talia Noreen

Lecturer

Beaconhouse National University

School of Computer and Information Technology (SCIT)

13 Km Niazbeg Raiwind Road (off Thokar, Beacon House Society

Lahore, Pakistan

E-mail: talia.noreen@bnu.edu.pk